Keylogging Detection at Kernel Level in Linux

Final Year Project Proposal by

Natasha Zaheer - 261934680
Muhammad Mamoon - 261938489
Haider Ali - 261941967

Advisor: Mr Muhammad Rauf Butt

Secondary Advisor: Dr Mubashar Mushtaq

Department of Computer Science
Forman Christian College University
Lahore, Pakistan (2025)

ABSTRACT

Keyloggers, especially those operating at the kernel level, represent one of the most sophisticated forms of cyber threats. Unlike user-space keyloggers, they execute with elevated privileges, enabling them to evade traditional detection mechanisms such as antivirus and heuristic-based tools. This project aims to develop a lightweight, AI-driven detection system that identifies kernel-level keylogging behavior in Linux environments through real-time analysis of system calls, kernel hooks, and process behaviors. The proposed system will integrate machine learning models trained on both benign and simulated malicious datasets to distinguish normal operations from potential keylogging activities with high accuracy and minimal system overhead. Additionally, the project includes a user-friendly visualization dashboard for real-time monitoring, alerts, and management of detected anomalies. Through kernel-level monitoring combined with interpretable AI models, the system will not only detect previously unseen threats but also provide transparent insights into the reasoning behind detections. By bridging the gap between academic anomaly detection research and practical Linux security tools, this project aspires to enhance the security and reliability of Linux-based systems against evolving cyber threats.

1. INTRODUCTION

Keyloggers running at the kernel level pose serious threats as they bypass user-space detection tools and operate with high privileges, making traditional signature or heuristic-based methods ineffective. To address this, our project proposes a lightweight, AI-driven monitoring system for Linux that uses machine learning on system call traces, kernel hooks, and process behavior to detect malicious keylogging in real time. The system adapts to new threats, provides clear insights through a user-friendly dashboard, and ensures minimal impact on performance, offering a practical and robust security solution.

2. PROBLEM STATEMENT

There is currently no widely adopted, lightweight, and interpretable kernel-level keylogger detection system that can accurately identify previously unseen threats in Linux environments. Existing solutions are either:

- User-space tools (**easily bypassed** by kernel-level keyloggers),
- Static/heuristic-based systems (**ineffective against evolving attacks**), or
- Heavy enterprise-grade monitoring platforms (too **resource-intensive** for lightweight Linux subsystems).

This leaves a significant **gap** in defending Linux systems—ranging from personal laptops to enterprise servers—against advanced keyloggers. The unmet need is for a real-time, kernel-level anomaly detection system that provides both accuracy and transparency while operating with minimal performance overhead.

Who needs it?

1. **System Administrators & Enterprises** – need a practical, lightweight tool to secure Linux servers, workstations, and critical infrastructure.
2. **Individual Linux Users & Developers** – privacy-conscious users, open-source developers, and security enthusiasts who require reliable protection against advanced threats on personal Linux-based systems.

3. LITERATURE REVIEW

Research on keylogger detection has grown substantially in recent years during the AI boom, but the work divides into two partially disconnected streams: (1) **AI/ML-based behavior analysis** (mostly user-space/trace-level research) and (2) **kernel-level monitoring & hardening** (tooling and recent research that observes events at the kernel boundary). To motivate the present project, it is important to synthesize both streams and highlight where they fail to overlap in ways that leave kernel-level keyloggers under-addressed.

AI and behavior-based detection (user space & classical ML)

Early and mid-range studies focused on learning behavioral signatures of keylogging or suspicious input capture using classical machine-learning algorithms and handcrafted features. Models such as Support Vector Machines, Random Forests, Naïve Bayes, and XGBoost were trained on features like API calls, process creation rates, keystroke timing, and file/network I/O; many reported high accuracy (often quoted in the 90–95% range) on their experimental datasets (Sahoo, S., Pradhan, R. & Mishra, D.P., 2024). Later work introduced sequence models (RNNs, LSTMs) able to reason about temporal order in system-call or keystroke streams, improving early detection at the cost of higher compute needs and reduced interpretability (Sahoo, Pradhan, & Mishra, 2024). Hybrid approaches—including bio-inspired algorithms (e.g., Dendritic Cell Algorithm combined with ML)—have shown promising detection rates in controlled settings (Chen & Wang, 2023). However, most of these studies use **user-space traces, simulated datasets, or small lab**

collections, which obscures how well the models generalize to *kernel-level* adversarial techniques (e.g., syscall table hooking, custom kernel modules, evdev interception).

Kernel-level monitoring & eBPF-based observability

Separately, the security community has developed kernel-level observability frameworks that make it feasible to collect rich telemetry without a full kernel module. The extended Berkeley Packet Filter (eBPF) ecosystem and practical tools like Tracee (Aqua Security) provide high-fidelity, low-overhead visibility into system calls, module loads, and other kernel events—capabilities that are directly relevant for detecting kernel-space manipulation and input interception (Aqua Security, Tracee). Tracee and related eBPF projects enable runtime security detection by observing kernel events in near real time, providing a practical telemetry substrate for ML models that require kernel-level signals. (aquasecurity, 2024)

Recent academic work has begun exploring ML directly on kernel traces. For example, kernel-level anomaly detection frameworks that analyze syscall patterns and module load behaviors demonstrate the feasibility of near real-time detection when kernel telemetry is available (Zhang & Wang, 2022). Other recent studies and preprints have proposed using eBPF to detect hidden rootkits and other kernel tampering, leveraging kernel event streams as ML input; however these works are nascent and often focus on prototype tools or detection heuristics rather than end-to-end, ML-driven detection pipelines validated across diverse Linux kernels and keylogger behaviors. Practical writeups and community articles also document techniques to capture kernel modules and rootkit indicators with Tracee and eBPF, underscoring broad industry interest in kernel observability for security.

Limitations & open problems (why more work is needed)

Despite the progress above, **important gaps** remain:

3.1 **Data gap at kernel granularity:** Many ML studies rely on user-space traces or small/simulated datasets. There is a shortage of publicly available, well-labeled datasets that contain *kernel-level traces of benign activity vs. kernel-space keylogger behaviors* (e.g., ioctl/evdev reads, module hook traces, kprobe events). This scarcity reduces

reproducibility and hinders comparison of ML approaches when the signal of interest lives in kernel space. Ongoing research exploring eBPF backdoors and the security/verification challenges of eBPF: eBPF's power is new, and the security community is actively researching both detection opportunities and risks. (K. Huang, 2025)

3.2 **Model portability & kernel diversity:** Kernel interfaces and behavior differ across distributions and kernel versions. Models trained on one kernel version may underperform on others, and research rarely reports cross-kernel generalization metrics.

3.3 **Runtime constraints & explainability:** Deep learning models that work well in user-space labs are often too heavy for low-overhead, real-time kernel-adjacent detection. Additionally, ML explainability (why an alert fired) is crucial for analyst triage but under-explored in kernel-level detection research. (Chen, J., & Wang, H., 2023)

3.4 **Evasion & adversarial robustness.** Kernel-space adversaries can manipulate hooks, throttle reads, or use obfuscation techniques to evade detectors. Research that combines kernel telemetry with robust ML/defense-aware modeling is limited and mostly exploratory.

3.5 **Tooling & safety tradeoffs.** While eBPF and related tooling (Tracee) make kernel telemetry accessible, they introduce their own risks (verification, memory-safety concerns, and anti-tampering limitations), and these tradeoffs are still actively studied (e.g., eBPF memory-safety analyses).

Taken together, these observations show that while both AI methods and kernel observability tools are advancing fast—especially in the **post-2022** period—there is **no mature, open, lightweight, interpretable AI system built and evaluated specifically for kernel-level keylogger detection across diverse kernels and realistic threat behaviors**. The combination of (a) building or collecting kernel-level datasets, (b) designing efficient interpretable ML models, and (c) integrating them with safe kernel telemetry (eBPF/tracee or a minimal kernel module) is therefore an open research and engineering problem.(aquasecurity, 2022)

How this project fits in

This project targets those exact gaps: we will (1) produce a labeled dataset of kernel-level traces including simulated but realistic keylogger behaviours (captured via safe kernel telemetry), (2) evaluate a spectrum of ML and anomaly-detection models with an emphasis on runtime footprint and explainability, and (3) demonstrate an integrated prototype (kernel telemetry → ML inference → dashboard) and compare it against baseline signature/heuristic detectors. By doing so we aim to contribute both a **practical artifact** and **research insights** into cross-kernel generalization, explainable detection, and operational tradeoffs for kernel-level AI security. Because much of the prior work and tooling is concentrated in the last 3 years, and because kernel attack techniques evolve rapidly, the research and engineering in this area remains timely and necessary.

4. PROJECT OVERVIEW/GOAL

4.1. Overview

Keyloggers at the kernel level pose severe security threats since they bypass user-space detection tools and operate with high privileges. Traditional solutions rely on behaviour-based or heuristic methods, which are often ineffective against advanced keyloggers, especially those operating at the kernel level.

Our project aims to design and implement such a system that leverages AI and runs at the kernel level to detect keyloggers, specifically for linux based subsystems. The system will leverage

machine learning models trained on system call traces, kernel hooks, and process behaviour tailored to each user to accurately differentiate between normal kernel activity and malicious keylogging operations.

Our solution differs from existing ones in the following ways:

- It can detect previously unseen keyloggers via **real-time anomaly detection.**
- It will be optimized for Linux, with minimal overhead on system performance.
- Integrating **interpretable ML models** to give insights into why a process was flagged as malicious on the dashboard, with the ability to mark the detection as acknowledged, or to quarantine the process.
- We will evaluate by deploying multiple real simulated keyloggers in a **controlled environment** and compare the results with existing **state-of-the-art** solutions.

Furthermore, documentation including datasets, ML model comparisons, and benchmarks will also be provided. Our project also contributes Linux keylogger traces in the form of a dataset to aid future research work, while introducing comparative evaluation with modern anomaly detection baselines.

This Final Year Project is both **research-driven** and **product-oriented** in nature. The research component focuses on exploring and developing novel AI-based methods for detecting kernel-level keyloggers in Linux by analyzing system call traces, kernel hooks, and process behaviors. The team will evaluate and benchmark different AI/ML models against state-of-the-art approaches, contributing to academic knowledge through dataset generation and comparative results.

At the same time, the project also has a strong **product focus**. The proposed solution will be implemented as a fully functional Linux application, featuring a kernel monitoring module integrated with an AI-driven anomaly detection engine and a real-time visualization dashboard. This ensures the system is not only theoretically sound but also practical, efficient, and user-friendly for real-world use by Linux users and system administrators.

By combining rigorous research with a deployable security product, this project bridges the gap between academic experimentation and practical cybersecurity tools — making it both **scientifically valuable and industrially relevant**.

`4.2 Goals

The goal is to develop a lightweight, real-time monitoring system for kernel activities using AI/ML models to detect anomalies that indicate keylogging behaviour. A user-friendly dashboard will be provided which visualizes detection, system health, and logs for security analysis. The performance will be compared against existing detection methods to prove higher accuracy and robustness against new/advanced keyloggers. Simulated keyloggers will be used that work on heuristic, behavioural and signature based detection techniques without an actual payload to work in a secure environment.

Instead of depending on static and existing solutions, which can easily be bypassed, our system will dynamically learn malicious behaviours. Unlike many user-space tools, our solution focuses on the kernel layer where advanced threats reside and disguise themselves. We aim to bridge the gap between academic anomaly detection research and practical Linux system security.

Our final output will be a visualization dashboard with the ability to manage detected threats, as well as the ability to tweak the AI/ML engine's permissions according to the user's own wishes. A linux kernel monitoring module with an AI detection engine will be provided along with the visualization dashboard in a packaged linux application that can be installed on any Linux subsystem.

5. PROJECT DEVELOPMENT METHODOLOGY / ARCHITECTURE

Our project follows a step-by-step approach to design, develop, and test a kernel-level AI system for keylogger detection in Linux. The work is divided into several modules, each focusing on a specific part of the system. Together, these modules will create a complete, real-time detection and visualization setup.

5.1 Project Methodology

5.1.1 **Kernel-Level Data Collection**

A custom Linux application will be developed which will use traces and tools like eBPF to monitor system calls and kernel hooks related to keyboard input and process activities. This module will capture important details such as which processes access the keyboard, how often they do it, and what system calls they make. These logs will form the main dataset for training the AI model.

5.1.2 **Data Preprocessing and Feature Extraction**

The collected data will then be cleaned and organized using Python. Only relevant features will be kept, such as call frequency, process behavior, and timing patterns. These features will help the model learn the difference between normal system activities and suspicious or malicious ones.

5.1.3 **AI/ML Model Development**

Different machine learning models such as Random Forest, Support Vector Machine (SVM), and Neural Networks will be trained and tested. The goal is to find the model that gives the highest accuracy with the least false alarms. Models will be trained on both normal and simulated malicious datasets created in a safe environment.

5.1.4 **System Integration**

Once the AI model and kernel module are ready, they will be integrated. The kernel module will continuously send live data to the AI engine, which will classify the behavior as safe or suspicious in real-time. The AI engine will be designed to run efficiently without slowing down the system.

5.1.5 **Dashboard Development**

A user-friendly dashboard will be created using Flask or another lightweight web framework. It will show alerts, process details, and detection results in an easy-to-understand format. Users will also be able to view system health, acknowledge detections, or quarantine suspicious processes.

5.1.6 **Testing and Evaluation**

The final system will be tested by running both normal applications and simulated keyloggers. The detection accuracy, processing speed, and resource usage will be compared with existing detection tools. This will help prove that our approach is lightweight, accurate, and practical for Linux users.

5.2 System Architecture

The overall architecture of the proposed system is divided into three main layers:

- Kernel Layer:

Responsible for capturing system-level events such as input access, process creation, and keyboard interaction.

- AI Detection Layer:

Processes the data received from the kernel, extracts features, and uses trained AI models to detect anomalies or keylogging behavior.

- Dashboard Layer:

Displays results visually through graphs, alerts, and logs, allowing the user to monitor system security in real-time.

6. PROJECT MILESTONES AND DELIVERABLES

Clear milestones and goals have been defined to guide the successful execution of the project.

Each milestone corresponds to an essential phase of development, from the initial research to the final development. This progression ensures systematic growth, regular evaluation, and measurable outcomes.

Milestones:

6.1. Literature Review and Problem Definition

Conduct a detailed review of existing detection mechanisms. Identify gaps in current methods and clearly define the problem scope for AI-based kernel-level detection of keyloggers.

6.2. System Architecture and Design

Define the proposed system architecture, including the kernel monitoring module, machine learning detection engine, and visualization dashboard. Outline the data flow between software components.

6.3. Dataset Collection and Preprocessing

Simulate multiple types of keyloggers in a controlled Linux environment (signature, heuristic, behavioral-based) and capture kernel-level traces. Prepare and preprocess these datasets for AI model training, ensuring security by avoiding active malicious payloads. Further, look at existing datasets and model work in the relevant field.

6.4. Model Development and Training

Develop and train machine learning models on the collected dataset to classify benign vs malicious kernel activities. Test multiple algorithms (Random Forest, SVM, Neural Networks, anomaly detection methods) and evaluate accuracy, recall, and false positive rates.

6.5. Kernel Module Implementation

Implement a lightweight Linux kernel module capable of intercepting input-related system calls and processes. Ensure low performance overhead and secure real-time data collection for feeding into the detection engine.

6.6. System Integration and Dashboard Development

Integrate the kernel module with the AI detection engine and build a user-friendly dashboard. The dashboard will display alerts, detected threats, system health, and provide options to acknowledge or quarantine flagged processes along with proper visualization of the AI pipeline with proper explanations for each detected vulnerability.

6.7. Testing, Benchmarking, and Comparison

Deploy multiple simulated keyloggers in the Linux testbed environment. Compare detection performance with existing approaches (heuristic, behavioral, signature-based). Conduct benchmarks to assess accuracy, detection speed, and system resource usage in comparison with existing static solutions.

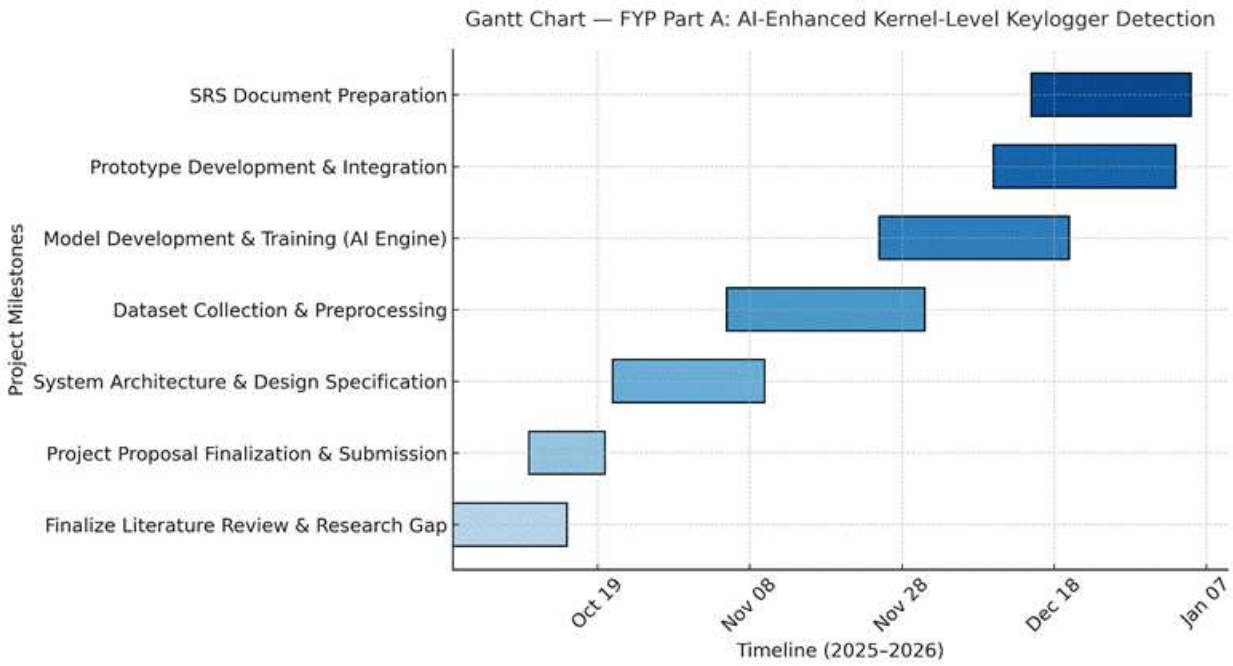
6.8. Final Packaging and Documentation

Deliver the complete packaged Linux application containing the kernel monitoring module, AI detection engine, and dashboard. Prepare comprehensive documentation, including datasets, model comparisons, evaluation benchmarks, and research contributions.

The final deliverables of this project will include a complete packaged Linux application consisting of a kernel monitoring module, an AI-based detection engine, and a user-friendly visualization dashboard. Alongside the software, a dataset of Linux kernel traces will be provided, containing both benign activities and simulated keylogging behavior, which can be used to aid future research in this domain. Comprehensive documentation will also be delivered, covering the system design, datasets, model evaluations, performance benchmarks, and comparative analysis with state-of-the-art approaches. A user manual will be included to guide installation, configuration, and day-to-day usage of the application. Finally, the project will culminate in a formal evaluation report

highlighting experimental results, along with a presentation and live demonstration to showcase the working prototype and its contributions to the field of kernel-level keylogger detection.

Gantt Chart



7. WORK DIVISION

Muhammad Mamoon	Preparation of Kernel Module + AI integration
Natasha Zaheer	Collection and preprocessing of data + model training
Haider Ali	Dashboard preparation + Kernel-AI Engine integration

8. COSTING

Cloud compute for ML model training + data preprocessing, certain subscription based linux kernel evaluation tools and modules.

REFERENCES

1. **Chen, J., & Wang, H. (2023). AI-based detection of keylogging attacks using system call traces. *IEEE Access, 11*, 10532–10547.**
2. **CrowdStrike. (2024). *Using artificial intelligence for real-time threat hunting and detection* (Whitepaper). Retrieved from https://www.crowdstrike.com/resources**
3. **Microsoft. (2023). *AI-driven threat detection in endpoint security* (Microsoft Security Blog). Retrieved from https://www.microsoft.com/security**
4. **Sahoo, S., Pradhan, R., & Mishra, D. P. (2024). A deep learning framework for detecting malicious keyloggers in Linux environments. *Journal of Cybersecurity, 10*(1).**
5. **SentinelOne. (2023). *Behavioral AI for endpoint detection and response*. SentinelLabs Technical Report. Retrieved from https://www.sentinelone.com/labs**
6. **Zhang, T., & Wang, Y. (2022). Kernel-level anomaly detection using machine learning. *Computers & Security, 114*, 102594. (Journal)**
7. **Aqua Security. (2024). Tracee — runtime security and forensics using eBPF. Retrieved from https://aquasecurity.github.io/tracee/ (Tracee is a practical eBPF-based runtime security tool widely used for kernel-level telemetry). aquasecurity.github.io**
8. **Aqua Security. (2022). Detecting and capturing kernel modules with Tracee and eBPF. *Aqua Security Blog*. Retrieved from https://www.aquasec.com/blog/linux-security-with-tracee-and-ebpf/. Aqua**
9. **K. Huang et al. (2025). SoK: Challenges and paths toward memory safety for eBPF. (analysis of risks and verification challenges in the eBPF ecosystem). Retrieved from https://nebelwelt.net/files/25Oakland.pdf. nebelwelt.net**
10. **Yu, Y. C. (2025). Kernel-level hidden rootkit detection based on eBPF. *Information & Computer Security* (article)**

SProj Proposal Defense Evaluation Form and Rubrics

Criteria	1	2	3	4	5
R1 Subject Knowledge	Student has no knowledge of both problem and solution. Cannot answer basic questions.	Student has no or very less knowledge of both problem and solution. Cannot answer questions.	Student is uncomfortable with information. Seems novice and can answer basic questions only.	Student has competent knowledge and is at ease with information. Can answer questions but without rationalization and explanation.	Student has presented full knowledge of both problem and solution. Answers to questions are strengthen by rationalization and explanation
R2 Organization and Content of	Student is clueless about the content of his presentation.	Information is arranged in confused and unstructured	Information articulated clearly but it is difficult to follow the presentation.	Information articulated clearly and the flow is reasonable	Information articulated clearly and is organized in a structured way with logical flow

Presentation		way. Key points are not covered. The contents are hard to understand and interpret.	All key points are covered but no use of charts, graphs, figures etc., to explain salient points.	All key points are covered but limited use of charts, graphs, figures etc., to explain salient points.	between parts. All key points are covered. Enhances presentation and keeps interest by effective use of charts, graphs, figures etc., to explain salient points.
R3 Problem Statement	Problem statement is not stated at all or vaguely stated Description of <u>unmet need</u> or problem is missing	Problem statement is stated but not entirely clear. Seems novice and can answer basic questions only.	Problem statement is stated but lacks necessary justification in light of the literature review.	Problem statement is stated and covers necessary justification with reference to the literature review. Details of the unmet need or problem the FYP is aiming to solve are clear	Problem statement is stated and covers sufficient justification. New reader can clearly understand its value and context. Details of <u>unmet needs</u> are there. <u>Potential customers</u> have been identified

R4 Literature Review	Literature Review is not written or written in a vague Form	Ligature Review is written in an ordinary way. The review material i.e. research papers or web material is not at all clear to a reader who is unfamiliar.	Literature review provides a reasonable description of the project background and its significance but can be improved. Number of research papers/ web material needs to be added more	The review provides a good background and details of the literature. However, it is not written in scientific writing standards for review.	Literature review is excellently written according to the scientific writing standards and covers maximum of the research papers/web material related to project

R5 Project Overview,	The approach that will be taken to solve	Some aspects of the solution are discussed	The methods, approaches, tools, techniques,	The methods, approaches, tools, techniques,	The methods, approaches, tools, techniques, algorithms, or other

4

Methodology	the problem is not discussed.	briefly but much of the description is left out.	algorithms, or other aspects of the solution are discussed but not is a convincing manner. Much is left to the readers' imagination.	algorithms, or other aspects of the solution are sufficiently discussed.	aspects of the solution are sufficiently discussed with sufficient details and supporting figures. Work division between group members is clearly defined

R6 Language and Grammar	A lot of spelling and grammatical mistakes in the report Writing is not understandable.	Frequent spellings and grammatical errors that impede the reading flow. Writing is in need of significant editing and improvement	Occasional spellings and grammatical errors Writing is acceptable but not entirely clear.	Occasional spellings and grammatical errors that have only minor impact on flow of reading. Writing is overall clear. Organization is good. Content is supported by good number of figures and tables.	Almost no spelling or grammatical mistake. Writing is easy to read. Excellent organization. Writing is concise yet all necessary content is included. Figures and tables support content.
R7 Delivery & Presentation Skills	Presentation was not clear at all. Language was not appropriate	Presenter occasionally spoke clearly. Holds little to no eye contact.	Presenter spoke clearly. Language was generally clear but mostly reading from notes.	Presenter spoke very clearly. Language was generally clear and delivery was fluent. Consistent use of direct eye contact with audience.	Presenter spoke clearly and at a good pace to ensure audience comprehension. Language was used effectively and delivery was fluent and expressive.

R8 Work Division	Work division among group members is not shown	Work Division among group members is not appropriate.	Work division is shown but more clarity is needed	Work division is shown.	Clear work division among group members is shown.